Structured output (a checked answer, not a hope) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `structured_output_practice.py`: asking for "JSON, please" in a prompt is only a request the model can ignore. A Pydantic model sent with `.parse()` is a form with locked fields: the model can only fill in the blanks you defined, in the types you defined, and the SDK hands you back an already-validated object. This exercise practices that form — first by hand, then with the model filling it. **If not:** the Build Task's `schemas.py`, Project 1's triage agent and Doc05's `.with_structured_output()` would be the first time you ever met a Pydantic model.

All examples below need `pip install openai python-dotenv pydantic` and your key in `.env`.

Solution

Story — `extract_info()`: the caller should never have to know about refusals, validation errors, or empty answers. This one function turns all three into a clean `None`, and everything else into an `ExtractedInfo` the caller can trust — one checkpoint instead of three scattered checks at every call site. **If not:** every caller would repeat the same three checks — and the one that forgets the refusal check reads `parsed` as `None` and crashes.

```
# structured_output_practice.py
from typing import Literal

from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, ValidationError

load_dotenv()
client = OpenAI()

SYSTEM_PROMPT = (
    "Pull the person's name, what they want, and how urgent it is out "
    "of the message. If there is no name or no request, use null — "
    "never guess."
)
```

```
class ExtractedInfo(BaseModel):
    # why: str | None, not str — a required name would force the model
    # to invent one when the message has none
    name: str | None
    # why: a message can ask for nothing at all ("the sky is blue")
    intent: str | None
    # why: Literal — the model can only pick one of these 3 exact values
    urgency: Literal["low", "normal", "urgent"]

# extract_info: the one function callers use. It turns refusals,
# validation errors and empty answers all into None, so the caller can
# trust whatever it gets back.
def extract_info(text):
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": text},
            ],
            response_format=ExtractedInfo,
        )
    except ValidationError:
        # when: the reply didn't fit the form — rare with .parse(), but
        # the caller should get "nothing", not a crash
        return None

    message = completion.choices[0].message
    # why: check refusal before parsed — when the model declines,
    # parsed is None and refusal holds the reason
    if message.refusal:
        print(f"The model refused: {message.refusal}")
        return None

    info = message.parsed
    # when: no name and no request — report "nothing found" instead of
    # handing back an object full of None
    if info.name is None and info.intent is None:
        return None
    return info

# --- First, see the validation itself, by hand ---
# model_validate() runs the same check .parse() runs on the model's JSON.
good = {"name": "Maria", "intent": "reset password", "urgency": "urgent"}
print(f"case 1 (good dict)    -> {ExtractedInfo.model_validate(good).urgency}")

bad = {"name": "Maria", "intent": "reset password", "urgency": "very"}
```

```
try:
    ExtractedInfo.model_validate(bad)
    print("case 2 (bad urgency)  -> accepted (wrong!)")
except ValidationError:
    print("case 2 (bad urgency)  -> ValidationError")

# --- Now let the model fill the form ---
texts = [
    "Hi, I'm Maria and I need my password reset today, please!",
    "Can someone check why my invoice is wrong?",
    "the sky is blue",
]
for text in texts:
    info = extract_info(text)
    print()
    print(f"Text: {text}")
    if info is None:
        print("  nothing to extract")
    else:
        print(f"  name:    {info.name}")
        print(f"  intent:  {info.intent}")
        print(f"  urgency: {info.urgency}")
```

Expected output (model values vary):

```
case 1 (good dict)    -> urgent
case 2 (bad urgency)  -> ValidationError

Text: Hi, I'm Maria and I need my password reset today, please!
  name:    Maria
  intent:  reset her password
  urgency: urgent

Text: Can someone check why my invoice is wrong?
  name:    None
  intent:  check a wrong invoice
  urgency: normal

Text: the sky is blue
  nothing to extract
```

The second text has no name, so `name` is `None` — the model reported "no name" instead of inventing one. That's what `str | None` plus the system prompt buys you.